

PASSWORD MANAGER APP



Software Project Plan

Table of Contents

1 - Introduction.....	2
1.1 - Purpose & Objectives	2
1.2 - Scope	3
1.3 - Definitions	3
1.4 - References	4
2 - Project Organization & Overview.....	5
2.1 - Team Contact	5
2.2 - Project Deliverables	5
2.3 - Project Milestones	5
2.4 - Git Branching	6
2.5 - Project Management	6
3 - Risk Analysis.....	7
3.1 - Credentials Security Risks	7
3.2 - Internet Connectivity Risks	8
3.3 - User Confusion Risks	9
4 - Software Resource Requirements.....	9
5 - Work Breakdown.....	11
6 - Project Schedule.....	14
7 - Monitoring and Testing.....	15
7.1 - Reporting	15
7.2 - Monitoring	16
7.3 - Testing	16

1. Introduction

1.1 Purpose & Objectives

Imagine a world where you never have to struggle to remember another password again, a world where your most sensitive credentials are locked away in an impenetrable vault, yet the vault is always available for you whenever you need it most. Well, this is not just a product of imagination anymore; this is the magic of GateKeep.

This document is your window into the journey of building GateKeep, a cutting-edge, cross-platform desktop password manager app that redefines what digital security and convenience should feel like, this document shows you the project organization (breakdowns of exactly what we are doing as well as who is doing each task), risk, project schedule, work breakdown structure, and the monitoring and testing part.

At its core, GateKeep isn't just about storing passwords, GateKeep is about empowering users to upgrade their security measures with the benefit of only remembering one single master password, GateKeep is about trust, ensuring users peace of mind that only the art of encryption can assure, protecting their most private information without them worrying about having to memorize all those annoying and long passwords.

1.2 Scope

GateKeep is not just another password manager. We're crafting a digital fortress specifically tailored for modern users who demand security but simplicity at the same time. Here is what GateKeep will bring to life:

- Fortress-Like Security : The user will be able to effortlessly store, retrieve, and manage their credentials in a heavily guarded, centralized vault cloud sync. Accessible with a master password.
- Effortless Access : Generate bulletproof strong passwords on the fly and watch as autofill works its magic across your system.
- Unbreakable Encryption : Built on AES-256 encryption and fortified with Argon2 hashing, GateKeep locks your data at all times.
- 2FA as Standard : From day one your account will always have 2 Factor-Authentication security.
- Cloud Sync Magic : Your data isn't bound to one single machine, no señor, GateKeep synchronizes your passwords across devices in real time no matter where you are at a highly secure level.

1.3 Definitions

- Git: Distributed version control system for tracking changes in source code.
- GitHub: Hosting platform for Git repositories and version control.
- GUI: Graphical User Interface.

- Open Source Software: Software that can be freely modified and shared under certain licenses.
- AES-256: Advanced Encryption Standard with 256-bit keys for data encryption.
- Argon2: Secure key derivation function used for password hashing.
- PostgreSQL: An open-source relational database system is used to store encrypted credentials.
- API: Application Programming Interface

1.4 References

- [How to Build a Password Manager in Python – GeeksforGeeks](#)
- [Lesspass: Stateless Open Source Password Manager](#)
- [Electron – Build Cross-Platform Desktop Apps](#)
- [Node.js — Run JavaScript Everywhere](#)
- [Azure Database for PostgreSQL](#)
- [PostgreSQL Documentation](#)
- [Argon2 – Wikipedia](#)
- [Credentials Management – Win32 apps | Microsoft Learn](#)
- [Keychain Services | Apple Developer Documentation](#)

2. Project Organization & Overview

2.1 Team Contact

Name	GitHub Profile	Email
Yash Patel	https://github.com/Yash1Patel	Ypatel5@pride.hofstra.edu
Josue Daniel Peraza	https://github.com/deadfeed2	Jperazanavarrete1@pride.hofstra.edu
Emmanuel Martinez	https://github.com/EmmanuelMar04	Emartinez5@pride.hofstra.edu
Leo Oliveira	https://github.com/LeotCode	Loliveira1@pride.hofstra.edu
Joseph Falco	https://github.com/JoeHofPasse	Jfalco15@pride.hofstra.edu
Derrick Casseus	https://github.com/DerrickCasseus	Dcasseus1@pride.hofstra.edu

2.2 Project Deliverables

The GateKeep Password Manager will be delivered as a cross-platform application built on Electron UI and C++ backend. It will be deployed in early Q2 of 2025 with the final presentation shown to Professor Jeffreys by May 1st, 2025. By this date, the final version of GateKeep will have all software requirements detailed in the requirements document.

2.3 Project Milestones

The first milestone for this application is UI development and backend API setup which have an estimated completion date of 3/6/25 and 3/16/25 respectively. This will have checked 3.3 and 5.1-5.5 in the requirements document. The next milestone is completing password creation/suggestion and encrypting those passwords which is outlined in 3.4 and 3.7. This task is

estimated to be done by 3/16/25 and 4/10/25 respectively. The third milestone is password strength checking and security alerts which are requirements 3.5, 4.1, and 4.3 to be completed by 3/1/25 and 3/11/25 respectively. The final milestone is database integration and browser/cloud sync, requirements 3.2, 3.6, and 4.2. This milestone is estimated to be completed by 4/3/25 and 4/18/25.

2.4 Git Branching

Our git branching policy will follow a structured layout to ensure project stability and code management. When making changes to git repos, specific guidelines will be followed to allow other team developers to understand what changes were made. This includes naming when branching and a small description of this new branch. This project will also have 2 main branches: a master branch with a release build and a development branch where members can fork off to work on their parts and add new features. For each new branch from the development branch, it will be named with the current feature in development and once finished it will be merged with the main development branch. Examples of branching are “new feature-hashing and encryption” and “bug fix- UI not refreshing”.

2.5 Project Management

The team is set to get on call once a week at 11 am every Saturday until the development of the project is completed. Everyone will provide regular updates on Discord. During the call, we will discuss how the progression of the software development is going and what tasks are upcoming. We will also confer about any changes to resource allocation and duration of tasks if any are needed.

3. Risk Analysis

GateKeep functions as a password storage application that's meant to be usable and trustworthy to the general public. However, this functionality comes with tons of security risks that could pose a massive threat to users if exploited. Users store passwords for all sorts of websites in GateKeep, so if a threat actor exploits one of these risks, any user can find their personal information compromised and to cause harm. This is why Jailhouse Lock takes full responsibility in determining what risks come with using GateKeep and mitigating both the likelihood of these risks being exploited alongside the damage that could be caused by these risks.

3.1 Credentials Security Risks

Due to GateKeep's purpose as a password storing application, it runs the risk of users having their passwords accessed by threat actors. GateKeep utilizes databases to store passwords so users can have them in one place to be easily viewed. However, the use of a database inherently runs the risk of others accessing those passwords since they need to be present and stored in order to be used. If these passwords are accessed by people who aren't meant to have them, the main user faces a high risk of losing their privacy and assets. However, Jailhouse Lock established multiple means to mitigate this risk.

Users create a master password that they need to use whenever they log on to a new device. Without this master password, these other devices cannot utilize a specific user's copy of GateKeep. Additionally, GateKeep comes with an automatic logout system for when users are away from their device for extended periods of time. This system hides the user's screen after a given period of time with a screen that only displays the option to log back into the application.

Through this, anyone who happens to pass by the user's screen won't be able to view any passwords the user may have been previously looking at.

However, these are not the only methods GateKeep utilizes to protect a user's passwords. Hidden from sight to regular users, GateKeep uses databases provided by PostgreSQL to contain all of a given user's passwords. This database is stored locally on a user's device, preventing an attack on one person's copy of GateKeep from affecting all GateKeep users. Additionally, the passwords stored within GateKeep are all encrypted with AES-256 to prevent those who can access the database from directly viewing the passwords. The key for these encryptions are also hidden to ensure that no one can easily view the raw passwords. GateKeep utilizes all of these methods to mitigate the risk of a user's passwords falling into the wrong hands.

3.2 Internet Connectivity Risks

Users can potentially face the risk of their passwords being intercepted when using GateKeep to automatically fill in passwords for a given website. Without a secure connection, threat actors could easily find out which passwords they use for a given website and exploit them from there. However, GateKeep mitigates these risks through the multiple APIs it has access to. GateKeep has Credential Manager API, Keychain API, and Secret Service API to help users of Windows, MacOS, and Linux respectively connect easily to any website. All three of these APIs provide a secure connection to the websites they have passwords for that fills in the expected password while lowering the chance that threat actors could intercept the connection.

3.3 User Confusion Risk

A smaller risk that users may face when using GateKeep is the possibility of users being confused about how they could use the app and accidentally do something they did not intend to

do. While this risk has a lesser impact than the other risks, users still need to understand GateKeep in order for it to properly fulfill its purpose. To help any users, GateKeep implements an intuitive user interface with clear directions to lead users to where they want to go in the app. To help guide users better, GateKeep provides users with the ability to search for passwords from a specific website so they won't have to worry about scrolling through all of their passwords.

4. Software Resource Requirements

Software resource requirements for GateKeep are the support software required to carry out the development of the application. Any failure to adhere to the requirements below may affect the estimated schedule.

Core Development Environment:

- Microsoft Visual Studio Code – Version 1.97.2 (or later) is mandatory as the primary code editor, offering flexibility, a rich ecosystem of extensions, and robust cross-platform support essential for GateKeep's multi-OS compatibility.

Required VSCode Extensions & Tools:

- C++ Extension: For building and compiling the core backend logic.
- Electron Extension: To facilitate cross-platform desktop application development.
- Python3 Extension: For scripting tasks, automation, and integration (especially for security features like Passlib.py).
- PostgreSQL Extension: To streamline database management, query execution, and integration within the codebase.

- Node.js Extension: To allow for the usage of custom javascript code on both ends of the client-server architecture.

Version Control & Collaboration:

- Git & GitHub: Every team member is required to have an active GitHub account. Git will serve as the backbone for version control, ensuring efficient collaboration and code integrity.

Development Protocols:

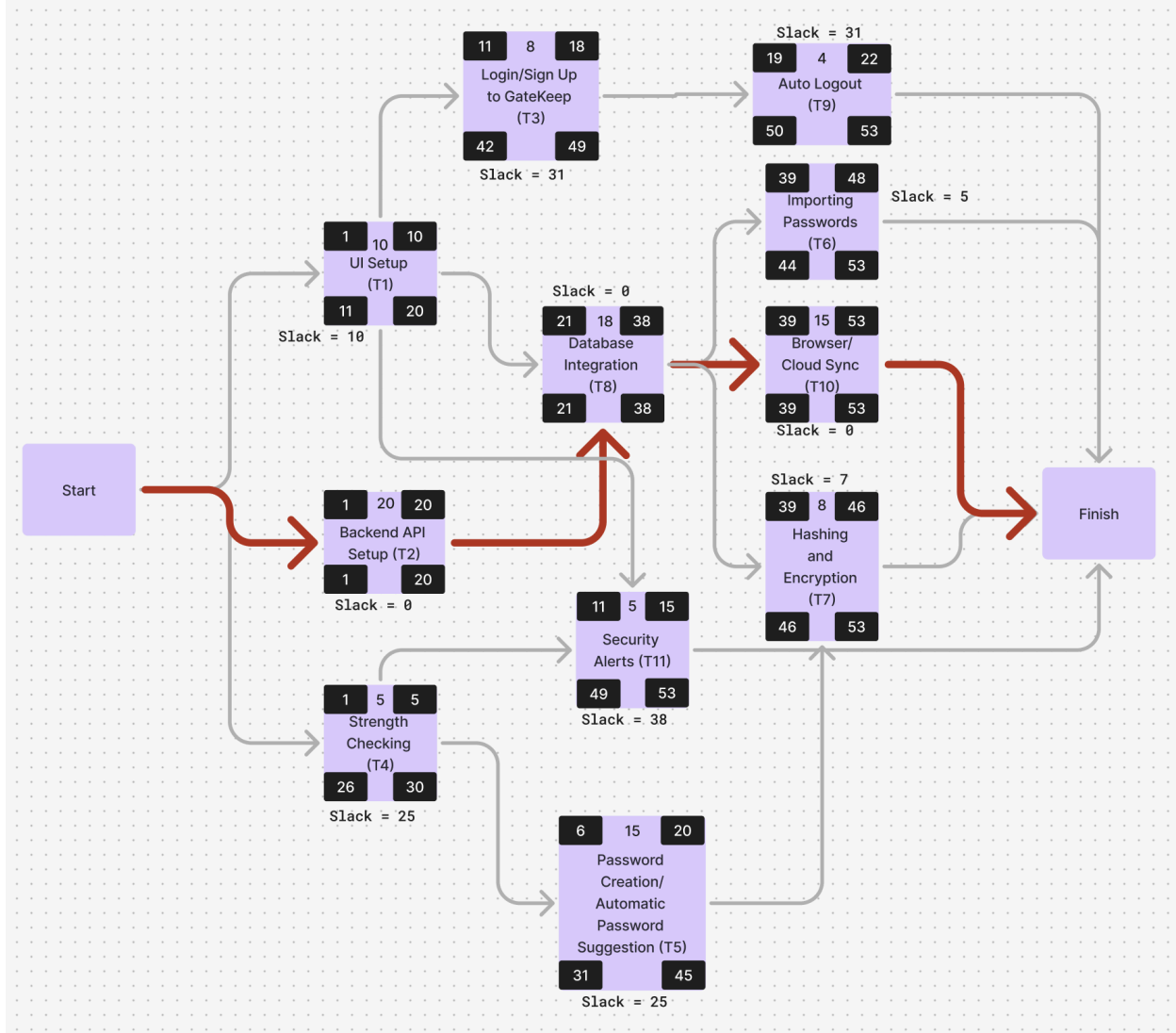
- Commit Regularly: All developers must upload, commit, and push code changes frequently to maintain code synchronization.
- Branching Strategy: Use feature branches for new development, and pull requests (PRs) for merging to the main branch.
- Code Reviews & Testing: Before merging, every PR must undergo peer review and pass all relevant tests to maintain code quality.
- Access Control: Developers should have the ability to check, test, modify, create, and delete files within their scope, ensuring smooth workflow management.

5. Work Breakdown

Tasks #	Task description	Effort (Days)	Risk	Dependencies	Resources (People)
T1 (Sections 5.1, 5.2, 5.3, 5.4, 5.5)	UI Setup	10	Medium	None	2
T2 (Section 3.3)	Backend API Setup	20	High	None	2
T3 (Sections 3.1, 4.1)	Login/Sign up to GateKeep	8	Medium	T1	2
T4 (Section 3.5)	Strength Checking	5	Medium	None	2
T5	Password Creation/Aut	15	High	T4	2

(Section 3.4)	o password suggestion				
T6 (Section 3.6)	Importing Passwords	10	High	T8	2
T7 (Section 3.7)	Hashing and Encryption	8	High	T5/T8	2
T8 (Sections 3.2, 4.2)	Database Integration	18	High	T1/T2	6
T9 (Section 3.1)	Auto Logout	4	Low	T3	2
T10 (Sections 3.2, 3.6)	Browser/Cloud Sync	15	High	T2/T8	4

T11	Security Alerts	5	Low	T1/T4	2
(Sections 4.1, 4.3)					

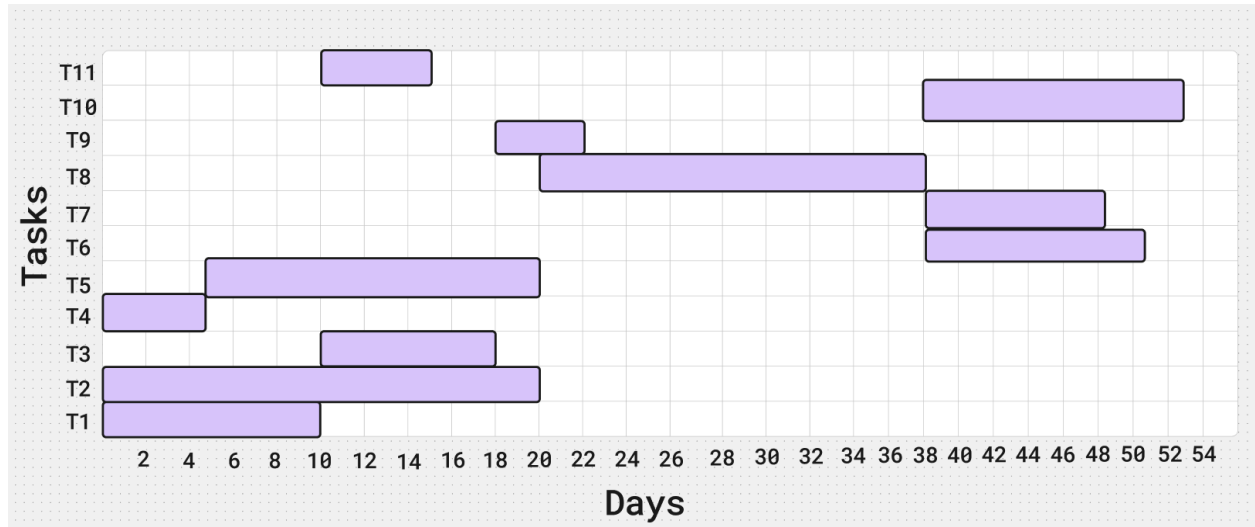


Critical path: T2 → T8 → T10

- Duration of critical path: 53 days

6. Project Schedule

The following Gantt chart represents the duration of the development period along with a schedule chart showing the start and end dates of each activity and their resource allocation. The official start date of the project is February 25th, 2025.



Activity	Start Date	Time (Days)	End Date	Resources
T1	2/25/25	10	3/6/25	Joe, Yash
T2	2/25/25	20	3/16/25	Emmanuel, Josue
T3	3/7/25	8	3/14/25	Joe, Yash
T4	2/25/25	5	3/1/25	Leo, Derrick
T5	3/2/25	15	3/16/25	Leo, Derrick

T6	4/3/25	10	4/12/25	Josue, Joe
T7	4/3/25	8	4/10/25	Yash, Leo
T8	3/17/25	18	4/3/25	Leo, Joe, Yash, Emmanuel, Josue, Derrick
T9	3/15/25	4	3/18/25	Joe, Yash
T10	4/4/25	15	4/18/25	Emmanuel, Josue
T11	3/7/25	5	3/11/25	Leo, Derrick

7. Monitoring and Testing

7.1 Reporting

The first progress report is due and will be presented in class on March 4, 2025. The second report will be shown on April 1, and the third report will be shown on April 17. The final product will be prepared by April 29 and presented in class on May 1st.

7.2 Monitoring

GateKeep is built every Friday at 9pm alongside when there is a new piece of code integrated into the program. Additionally, the GitHub repository is checked every 48 hours to make sure everything is in line with the current expectations for GateKeep's progress.

7.3 Testing

GateKeep is tested for compatibility with both Windows and Mac devices. GateKeep is developed to where both systems would share the exact same bugs. The developers perform integration testing through analyzing and testing their developed code on their own before submitting it into the GitHub. Upon submission, the other developers are to test out the code to ensure there wasn't any missed code. In the case where a bug is discovered, the original developers are informed of it so proper alterations to the original code are made. Common data used in tests were test accounts and custom passwords since they are essential information required for GateKeep to function. Additional forms of testing done are regression, unit, and system testing whenever required to ensure the highest quality for GitHub. Testing is done up until the final day possible by the team.